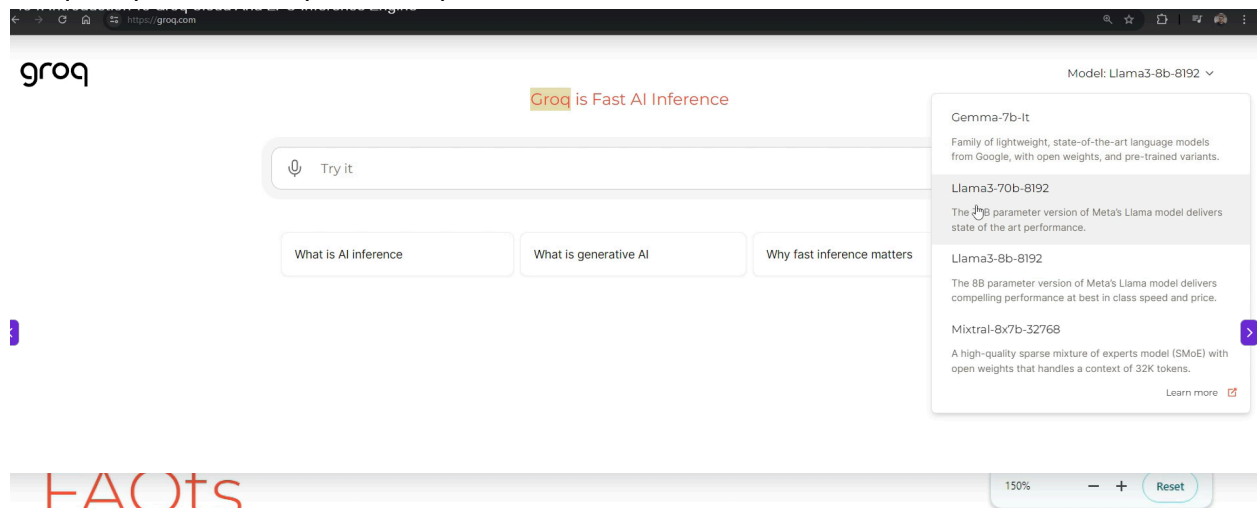


Groq is a platform that provides opensource LLM models



+ What is Groq?

- What is the LPU™ Inference Engine?

An LPU Inference Engine, with LPU standing for Language Processing Unit™, is a hardware and software platform that delivers exceptional compute speed, quality, and energy efficiency. This new type of end-to-end processing unit system provides the fastest inference for computationally intensive applications with sequential components, such as AI language applications like Large Language Models (LLMs).

+ Why is it so much faster than GPUs for LLMs and GenAI?

+ Does Groq run standard machine learning frameworks?

+ How do I get started using Groq?

- What is the LPU™ Inference Engine?

- Why is it so much faster than GPUs for LLMs and GenAI?

The LPU is designed to overcome the two LLM bottlenecks: compute density and memory bandwidth. An LPU has greater compute capacity than a GPU and CPU in regards to LLMs. This reduces the amount of time per word calculated, allowing sequences of text to be generated much faster. Additionally, eliminating external memory bottlenecks enables the LPU Inference Engine to deliver orders of magnitude better performance on LLMs compared to GPUs.

For a more technical read about our architecture, download our ISCA-awarded 2020 and 2022 papers.

create an api key

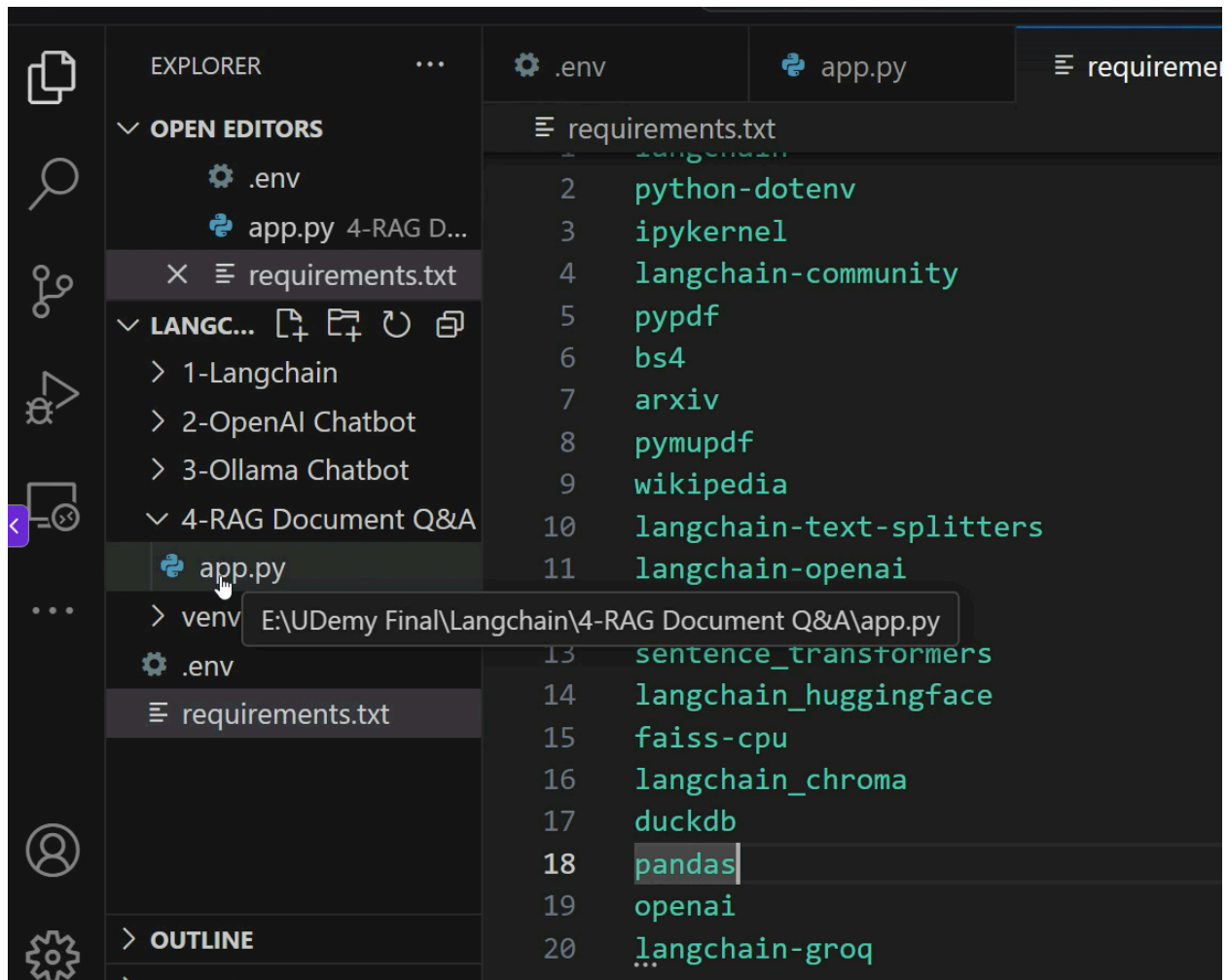
The image shows two screenshots related to setting up a Groq API key for a project.

The top screenshot is a browser window at <https://console.groq.com/keys>. The page title is "API Keys". It contains the text: "Manage your API keys. Remember to keep your API keys safe to prevent unauthorized access." Below this is a table with headers: NAME, SECRET KEY, CREATED, and LAST USED. The table is currently empty. A "Create API Key" button is visible at the bottom of the table area.

The bottom screenshot is a VS Code editor window titled "164. Introduction To Groq Cloud And LPU Inference Engine". The Explorer sidebar on the left shows the project structure with folders for "OPEN EDITORS", "LANGCHAIN", and "4-RAG Document Q&A". The ".env" file is selected and open in the editor. The content of the ".env" file is as follows:

```
1 OPENAI_API_KEY="sk-proj-qCMZ5GVbPZv0rxABfmd6T3B1bkFJ8Y1FI9sWj2qaWTBronR"
2 LANGCHAIN_API_KEY="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"
3 LANGCHAIN_PROJECT="GenAIAPPWithOPENAI"
4 HF_TOKEN="hf_MUGhWGrUJTfabjqwaXUuRXpyYmElzWlOC1"
5 HUGGINGFACEHUB_API_TOKEN="hf_MUGhWGrUJTfabjqwaXUuRXpyYmElzWlOC1"
6 GROQ_API_KEY="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"
7
```

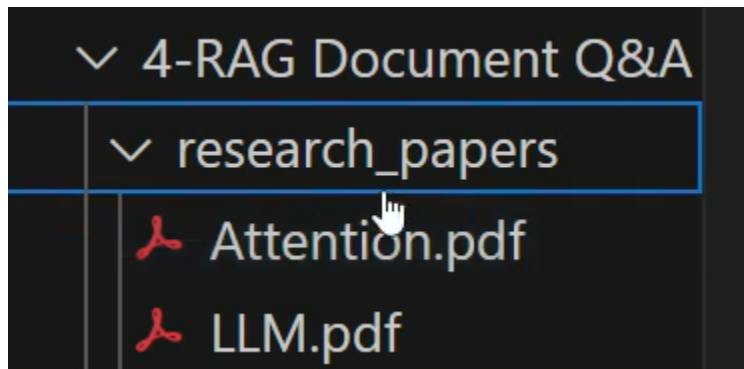
add package langchain-groq



4-RAG Document Q&A > app.py

```
1 import streamlit as st
2 import os
3 from langchain_groq import ChatGroq
4 from langchain_openai import OpenAIEmbeddings
5 from langchain_community.embeddings import OllamaEmbeddings
6 from langchain.text_splitter import RecursiveCharacterTextSplitter
7 from langchain.chains.combine_documents import create_stuff_documents_chain
8 from langchain_core.prompts import ChatPromptTemplate
9 from langchain.chains import create_retrieval_chain
10 from langchain_community.vectorstores import FAISS
11 from langchain_community.document_loaders import PyPDFDirectoryLoader
```

here we have 2 amazing research folders that act as external datasource.



```
from dotenv import load_dotenv
load_dotenv()
## load the GROQ API Key
os.environ['GROQ_API_KEY']=os.getenv("GROQ_API_KEY")

groq_api_key=os.getenv("GROQ_API_KEY")

llm=ChatGroq(groq_api_key=groq_api_key,model_name="Llama3-8b-8192")
```

OllamaEmbeddings usually refers to a feature/class used to generate **text embeddings** using local models via Ollama.**

What are embeddings (quick refresher)

Embeddings are just:

👉 **Numbers (vectors) that represent meaning of text**

For example:

- “dog” and “puppy” → very similar vectors
- “dog” and “car” → very different vectors

They’re used for:

- Semantic search
- Recommendations
- Chat memory / retrieval (RAG)
- Clustering text

What **OllamaEmbeddings** does

OllamaEmbeddings lets you:

👉 Generate these vectors **locally on your machine** instead of calling a cloud API

So instead of using something like:

- OpenAI embeddings API

You use:

- Ollama + an embedding model (like **nomic-embed-text**, **mxbai-embed-large**, etc.)
-

Where you see it

You'll commonly see **OllamaEmbeddings** in frameworks like:

- LangChain
- LlamaIndex

Example (Python-style):

```
from langchain.embeddings import OllamaEmbeddings
```

```
embeddings = OllamaEmbeddings(model="nomic-embed-text")  
vector = embeddings.embed_query("What is AI?")
```

vs OpenAI embeddings

Feature	OllamaEmbeddings	OpenAI Embeddings
Runs where	Local	Cloud
Cost	Free (no tokens)	Pay per token
Privacy	Full (local)	Data sent to API

Speed	Depends on your machine	Usually fast
Quality	Good (model-dependent)	Very strong

When to use it

Use `OllamaEmbeddings` if you want:

-  Full privacy (local data)
-  No API cost
-  Experimenting with RAG locally

Avoid it if:

- You need **top-tier embedding quality**
 - Your machine is weak (embeddings can still be compute-heavy)
-

Bottom line

`OllamaEmbeddings` =

👉 **Local alternative to OpenAI embeddings**

👉 Same concept, different execution (your machine vs cloud)

If you want, I can show you a **full RAG pipeline using Ollama embeddings + a local LLM** so you see how it fits together.

refer to the code, it is clear

```
def create_vector_embedding():
    if "vectors" not in st.session_state:
        st.session_state.embeddings=OllamaEmbeddings()
        st.session_state.loader=PyPDFDirectoryLoader("research_papers") ## Data Ingestion s
        st.session_state.docs=st.session_state.loader.load() ## Document Loading
        st.session_state.text_splitter=RecursiveCharacterTextSplitter(chunk_size=1000,chunk
        st.session_state.final_documents=st.session_state.text_splitter.split_documents(st.
        st.session_state.vectors=FAISS.from_documents(st.session_state.final_documents,st.s
```

if possible refer to the video, the way he implemented, but may not be needed on revision. just look into the code and if you don't understand you can review the video.

difference between langchain,groq, and ollama

This comparison is tricky because **these three are NOT the same type of tool**. If you try to compare them directly, it's like comparing:

👉 *a framework vs a runtime vs a cloud engine*

Let's break it down clearly.

Core idea (one-line each)

- **LangChain** → *Builds AI apps (logic layer)*
- **Ollama** → *Runs models locally (execution layer)*
- **Groq** → *Runs models in ultra-fast cloud (execution layer)*

Side-by-side comparison

Feature	LangChain	Ollama	Groq
Type	Framework	Local runtime	Cloud inference API
Main role	Build workflows (RAG, agents)	Run LLMs locally	Run LLMs fast in cloud

Where models run	Anywhere (via APIs)	Your machine	Groq servers
Speed	Depends on backend	Medium (hardware-limited)	⚡ Extremely fast
Cost	Free (framework)	Free (local)	Pay-per-use
Privacy	Depends on backend	High (local)	Lower (cloud)
Ease of use	Medium	Easy	Easy (API)

What each one actually does

1. LangChain (the “brain builder”)

- Helps you **build AI applications**
- Adds:
 - Chains (multi-step workflows)
 - Agents (tool-using AI)
 - Memory
 - RAG pipelines

👉 It **does NOT** run models itself

👉 It connects to models (OpenAI, Ollama, Groq, etc.) ([Leanware](#))

Think:

“How do I structure my AI app?”

2. Ollama (the “local engine”)

- Runs open-source models on your machine

Simple commands like:

ollama run llama3

-
- No API cost, full privacy

👉 Focus = **local inference + model management** ([Easy Python](#))

Think:

“Run AI locally without cloud”

3. ⚡ Groq (the “speed monster”)

- Cloud API for running LLMs
- Uses special hardware (LPUs) for **very fast inference** ([AI Agent Store](#))
- Much faster than typical APIs

👉 Focus = **speed + scalability**

Think:

“Run AI extremely fast in the cloud”

⚡ Real-world difference (important)

Same task → different tools

If you build a chatbot:

- **LangChain** → builds the chatbot logic
- **Ollama** → runs the model locally
- **Groq** → runs the model super fast in cloud

👉 They are **complementary**, not competitors

🧩 How they work together

Typical stack:

LangChain (logic)

↓

Groq OR Ollama (model execution)

Example:

- LangChain → handles RAG pipeline
 - Ollama → runs local LLM
 - OR Groq → runs fast cloud LLM
-

Trade-offs (honest view)

Ollama

-  Free, private
-  Slower, hardware-limited

Groq

-  Extremely fast
-  Costs money, cloud dependency

LangChain

-  Powerful abstraction
 -  Can be complex / overkill for simple apps ([Leanware](#))
-

Simple analogy

- **LangChain** = Chef (decides recipe)
 - **Ollama** = Your home kitchen
 - **Groq** = Industrial high-speed kitchen
-



When to use what

- Use **LangChain** if:
 - You're building RAG, agents, workflows
 - Use **Ollama** if:
 - You want **local + private AI**
 - No API costs
 - Use **Groq** if:
 - You want **speed + production scale**
-



Bottom line

- **LangChain** = “how to build AI apps”
 - **Ollama** = “run AI locally”
 - **Groq** = “run AI very fast in cloud”
-

If you want, I can show a **real architecture (code-level)** combining all three so it clicks instantly.

When you compare above with hugging face, here is the answer

Hugging Face is yet another piece of the AI stack—and again, it's not the same type of thing as the others. The clean way to understand it:

👉 **Hugging Face = model hub + tools ecosystem**



One-line comparison (all four)

- **LangChain** → builds AI app logic
 - **Ollama** → runs models locally
 - **Groq** → runs models super fast in cloud
 - **Hugging Face** → **finds, hosts, and shares models + tools**
-



Side-by-side comparison

Feature	LangChain	Ollama	Groq	Hugging Face
Type	Framework	Local runtime	Cloud inference	Model hub + platform
Main role	Build workflows	Run locally	Run fast in cloud	Discover & use models
Hosts models?	✗	✓ (local)	✗	✓ (huge library)
Runs models?	✗	✓	✓	✓ (via inference endpoints)
Open-source focus	Medium	High	Medium	Very high
Cost	Free	Free	Paid	Free + paid options



What Hugging Face actually does

1. 📦 Model Hub (biggest feature)

- Thousands of models:
 - LLMs (Llama, Mistral, etc.)
 - Embedding models
 - Vision, audio, etc.

👉 Think: “**GitHub for AI models**”

2. 🛠 Libraries

- `transformers` (run models)
- `datasets`
- `tokenizers`

👉 Lets you run models in Python directly

3. Inference API (optional)

- Run models in the cloud (like OpenAI/Groq)
 - No local setup needed
-

How it connects to others

This is where it clicks:

With Ollama

- You often **download models from Hugging Face**
- Then run them locally via Ollama

👉 Hugging Face = source

👉 Ollama = execution

With LangChain

- LangChain can use Hugging Face models
 - For embeddings, LLMs, etc.
-

With Groq

- Groq may serve optimized versions of models that originated on Hugging Face
-

Real-world flow

Example:

Hugging Face → get model



Ollama → run locally

↓
LangChain → build app logic

OR

LangChain → logic

↓
Groq → fast cloud inference



Simple analogy

- **Hugging Face** = App Store (models)
 - **Ollama** = Your phone (runs apps locally)
 - **Groq** = Cloud supercomputer (runs apps fast)
 - **LangChain** = App developer toolkit
-



Key differences (important)

- Hugging Face is **not just one thing**
 - It's **models + tools + hosting**
- It overlaps a bit with:
 - Ollama (both can run models)
 - Groq (both offer cloud inference)

But:

👉 Hugging Face is more **ecosystem + distribution platform**



When to use Hugging Face

Use it if you want:

- Access to **open-source models**
- Fine-tune or experiment
- Build custom ML pipelines

- Download embedding/LLM models
-



Final takeaway

- **LangChain** → builds apps
 - **Ollama** → runs locally
 - **Groq** → runs fast in cloud
 - **Hugging Face** → provides the models + tools
-

If you want, I can map a **complete beginner** → **advanced roadmap** using all four so you know what to learn first.

Core building blocks

- **LLMs** (models that generate text)
- **Embeddings** (convert text → vectors)
- **Vector stores** (store/search embeddings)
- **Retrievers** (fetch relevant data)
- **Prompts / templates**
- **Chains / pipelines**
- **Agents** (tool-using AI)

Those terms can mean slightly different things depending on where you saw them, but in the **LLM / GenAI ecosystem** (LangChain, Hugging Face, etc.) they usually map to two big ideas:



1. “Building blocks” (what it means)

Building blocks = reusable components you combine to build AI apps

Think of them as LEGO pieces.

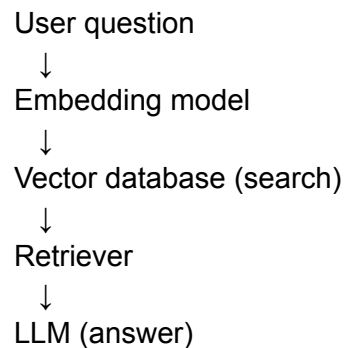
In frameworks like LangChain or ecosystems like Hugging Face, building blocks include things like:

- ♦ **Core building blocks**

- **LLMs** (models that generate text)
 - **Embeddings** (convert text → vectors)
 - **Vector stores** (store/search embeddings)
 - **Retrievers** (fetch relevant data)
 - **Prompts / templates**
 - **Chains / pipelines**
 - **Agents (tool-using AI)**
-

Example (how blocks fit together)

A simple RAG app:



👉 Each step = a **building block**

Why this matters

Instead of writing everything from scratch, you:

- Plug components together
 - Swap parts easily (e.g., change model from Ollama → Groq)
-

2. “Context hub” (what it means)

This is a **less standardized term**, but generally:

👉 **Context hub = a central place/system that manages all the context your AI uses**

What “context” means

Context = everything the model needs to answer well:

- User input
- Chat history
- Retrieved documents (RAG)
- System instructions
- External data (APIs, DBs)

What a context hub does

A context hub typically:

- Collects data from multiple sources
- Filters what’s relevant
- Formats it for the model
- Sends it as input (prompt/context window)

Where you see this idea

In LangChain

- Memory modules
- Retrievers
- Prompt assembly

👉 Together, they act like a **context hub**

In LlamaIndex

- Focus is heavily on **context management**
 - Indexing + retrieval = core “context hub” behavior
-

In production systems

People often build a **custom context layer** that:

- Pulls from databases
 - Uses embeddings
 - Maintains conversation state
-

Simple analogy

- **Building blocks** = ingredients (flour, sugar, eggs)
 - **Context hub** = the mixing bowl where everything comes together
-

How they relate

- Building blocks = individual components
 - Context hub = how those components **work together to supply context to the model**
-

Bottom line

- **Building blocks** → pieces used to build AI systems
 - **Context hub** → system that gathers & manages all input/context for the model
-

If you tell me where you saw “context hub” (course, tool, doc), I can give a more precise explanation—it’s one of those terms people use slightly differently depending on the stack.

What is retriever

Retrievers are a core concept in RAG systems.

👉 **Simple definition:**

A **retriever** finds the *most relevant pieces of data* for a given question.

Think of it like this

You ask:

“What is the refund policy?”

The retriever:

- Searches your documents
- Picks the **most relevant chunks**
- Sends them to the LLM

👉 It does **search, not answering**

Where retrievers fit

In a RAG pipeline (like your code using LangChain):

User Question



Retriever ← (this part)



Relevant Chunks



LLM (e.g., via `:contentReference[oaicite:1]{index=1}`)



Final Answer

How retrievers work

Most modern retrievers use:

1. Embeddings

- Convert text → vectors
- (e.g., via Hugging Face models)

2. Similarity search

- Compare your question vector
 - With stored document vectors
 - Return closest matches
-

Example

Documents:

- “Refunds are allowed within 30 days”
- “Shipping takes 5 days”

User asks:

“Can I return a product?”

Retriever returns:

👉 “Refunds are allowed within 30 days”

Types of retrievers

1. Vector retriever (most common)

- Uses embeddings + similarity
 - Example: `Chroma.as_retriever()`
-

2. Keyword retriever

- Matches exact words (like Google search)
-

3. Hybrid retriever

- Combines both (better accuracy)
-

4. History-aware retriever (you used this!)

- Uses chat history to rewrite queries
 - Fixes vague questions like:
 - “What about pricing?”
-



In your code specifically

```
retriever = vectorstore.as_retriever()
```

👉 This:

- Wraps your vector DB (Chroma)
 - Turns it into a retriever
-



Important distinction

- **Retriever ≠ LLM**
- Retriever → finds info

- LLM → explains / generates answer
-



Simple analogy

- Retriever = librarian 
- LLM = teacher 

You ask a question:

- Librarian finds the right books
 - Teacher explains the answer
-



Bottom line

👉 **Retrievers = smart search layer in AI apps**

They decide *what information the model sees*

If you want, I can show:

- 🔥 how to improve retriever accuracy
- ⚡ or visualize similarity search step-by-step

What is hugging face embeddings

Good question—this is a **very common point of confusion**.

👉 Short answer:

- **AIEmbeddings** = generic / abstract concept (or base interface)
 - **HuggingFaceEmbeddings** = specific implementation using Hugging Face models
-



Think of it like this

- **AIEmbeddings** → “any embedding system”
 - **HuggingFaceEmbeddings** → “one specific embedding provider”
-



Clear comparison

Feature	AIEmbeddings	HuggingFaceEmbeddings
Type	Abstract concept / base class	Concrete implementation
Purpose	Define <i>how embeddings should work</i>	Actually generate embeddings
Runs where	Depends	Local or HF API
Model	Not specified	e.g. <code>all-MiniLM-L6-v2</code>
Usable directly?	❌ No	✅ Yes



What is **AIEmbeddings** really?

In frameworks like LangChain:

👉 **Embeddings** (sometimes casually called “AIEmbeddings”) is:

- A **base interface / class**

Defines methods like:

`embed_query(text)`

`embed_documents(texts)`

-

It does NOT:

- Choose a model
- Generate vectors

👉 It's just a **template**

What is **HuggingFaceEmbeddings**?

```
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

This:

- Loads a real embedding model
 - Converts text → vectors
 - Can run locally
-

Other embedding implementations

Besides Hugging Face, you can plug in:

- OpenAI embeddings
- Ollama embeddings
- Cohere embeddings

👉 All follow the same **Embeddings interface**

Why this design exists

Because LangChain is modular:

You can swap:

```
# Hugging Face  
HuggingFaceEmbeddings()
```

```
# OpenAI  
OpenAIEmbeddings()
```

```
# Ollama
OllamaEmbeddings()
```

👉 without changing the rest of your code

Simple analogy

- **AIEmbeddings** = USB port
 - **HuggingFaceEmbeddings** = actual USB device
-

Important note

“AIEmbeddings” is often:

- A **misused term**
- Or shorthand people use for “embeddings in general”

👉 In real code, you’ll almost always use a **specific class** like:

- **HuggingFaceEmbeddings**
-

Bottom line

- **AIEmbeddings** → concept / interface
 - **HuggingFaceEmbeddings** → working implementation
-

If you want, I can show a **side-by-side code swap (Hugging Face ↔ OpenAI ↔ Ollama embeddings)** so you see how interchangeable they are.